

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



BIG DATA

Midterm Report

Building a Hybrid Search System for Academic Documents using Elasticsearch

Instructor(s): PGS.TS Thoai Nam
Student(s): Duong Gia An 2470293
Trinh Son Lam 2470287
Nguyen Thanh Huyen 2570416

HO CHI MINH CITY, MARCH 2026

Contribution

Team Effort Breakdown:

Member	Name	Contribute	Effort (%)
1	Duong Gia An	System Architecture, Elasticsearch Setup	33.33%
2	Trinh Son Lam	Data Cleaning, Problems & Solutions	33.33%
3	Nguyen Thanh Huyen	Theoretical Background, Report Formatting	33.33%

Mục lục

Contribution

List of Figures	i
---------------------------	---

1 Giới thiệu (Introduction)	1
1.1 Bối cảnh vấn đề	1
1.2 Mục tiêu dự án	2
1.3 Phạm vi (Scope)	2
2 Dataset (Dữ liệu)	4
2.1 Nguồn dữ liệu	4
2.2 Quy mô và Đặc điểm	4
2.3 Cấu trúc dữ liệu khai thác	4
2.4 Quy trình làm sạch dữ liệu (Data Cleaning)	5
3 Vấn đề và Giải pháp (Problems & Proposed Solutions)	7
3.1 Problem 1: Sự cứng nhắc của tìm kiếm từ khóa (The Rigidity of Keyword Search)	7
3.2 Problem 2: AI bỏ sót các thuật ngữ chuyên ngành (AI Misses Specialized Terminology)	7
3.3 Problem 3: Nhiễu tìm kiếm đa ngành (Search Skewed by Multidisciplinary Noise)	7
4 Cơ sở lý thuyết (Theoretical Background)	9
4.1 Information Retrieval Pipeline	9
4.2 Inverted Index và Thuật toán BM25	9
4.3 Field Types trong Elasticsearch	10
4.4 Vector Embedding và k-Nearest Neighbors (kNN)	11
4.5 Reciprocal Rank Fusion (RRF)	11
5 Kiến trúc hệ thống (System Architecture)	12
5.1 Sơ đồ luồng dữ liệu (Data Flow)	12
5.2 Kiến trúc hạ tầng (Infrastructure)	13
5.2.1 Công nghệ sử dụng	13
5.2.2 Mô hình triển khai	13
5.2.3 Đặc tính Big Data	14

5.3	Index Mapping & Bulk Strategy	14
5.3.1	Mapping Definition	14
5.3.2	Bulk Indexing Strategy	15
6	Hiện trạng triển khai (Current Progress)	16
6.1	Các hạng mục đã hoàn thành	16
6.1.1	Thiết lập hạ tầng (Infrastructure Setup)	16
6.1.2	Tiền xử lý dữ liệu (Data Engineering)	16
6.1.3	Lập chỉ mục (Bulk Indexing)	17
6.1.4	Truy vấn cơ sở (Baseline BM25 Search)	17
6.2	Các hạng mục tiếp theo (Next Steps)	20
7	Kết luận tạm thời	22
7.1	Tóm tắt kết quả đạt được	22
7.2	Đánh giá tính khả thi	22
7.3	Bước tiếp theo	22
	References	24

List of Figures

5.1.1 Sơ đồ luồng dữ liệu (Data Flow Pipeline) của hệ thống	12
6.1.1 Docker containers đang chạy Elasticsearch 8.12 và Kibana	16
6.1.2 Kibana Dev Tools – Kết quả xác nhận 100,000 documents đã được index	17
6.1.3 Kibana Dev Tools – Kết quả tìm kiếm BM25 với filter	20

Chương 1

Giới thiệu (Introduction)

1.1 Bối cảnh vấn đề

Trong những năm gần đây, sự phát triển mạnh mẽ của các nghiên cứu khoa học đã tạo ra một sự bùng nổ về khối lượng tài liệu học thuật. Các kho lưu trữ mở như arXiv hiện đang chứa hàng triệu bài báo trải dài trên hàng trăm lĩnh vực khác nhau, tạo thành một nguồn dữ liệu khổng lồ (Big Data). Mặc dù sự phong phú này là một tài nguyên quý giá, nhưng nó cũng trực tiếp dẫn đến bài toán "quá tải thông tin". Sinh viên và các nhà nghiên cứu đang phải dành một lượng thời gian khổng lồ chỉ để tra cứu, sàng lọc và đánh giá mức độ liên quan của tài liệu, thay vì tập trung vào việc nghiên cứu cốt lõi.

Đứng trước khối lượng dữ liệu khổng lồ này, các công cụ tìm kiếm học thuật truyền thống bắt đầu bộc lộ sự quá tải và những giới hạn về mặt kiến trúc.

- Nếu áp dụng các công cụ tìm kiếm thuần dựa trên từ khóa (Lexical Search), hệ thống sẽ vô cùng cứng nhắc, không thể hiểu được ngữ cảnh phức tạp, từ đồng nghĩa hay ý đồ thực sự của người tra cứu.
- Ngược lại, nếu chỉ lạm dụng các mô hình Trí tuệ nhân tạo (AI Search) để hiểu ngữ nghĩa, hệ thống lại dễ dàng đánh mất tính chính xác tuyệt đối—một yếu tố sống còn trong nghiên cứu khoa học khi người dùng cần tìm đích danh một tác giả cụ thể, một công thức, hay một thuật ngữ viết tắt chuyên ngành hiếm gặp. Hơn nữa, việc áp dụng các bộ lọc siêu dữ liệu (như năm xuất bản, danh mục) lên các hệ thống AI thường xuyên gây ra sự sai lệch hoặc làm giảm hiệu suất hệ thống đáng kể.

Với lượng dữ liệu lớn nhưng thiếu công cụ trích xuất thông minh đã đặt ra một yêu cầu cấp thiết: Cần phải có một kiến trúc tìm kiếm thế hệ mới, có khả năng xử lý Dữ liệu lớn với tốc độ cao, đồng thời dung hòa được cả độ chính xác của tìm kiếm từ khóa và sự thông minh của AI hiểu ngữ cảnh [5].

1.2 Mục tiêu dự án

Dự án hướng tới việc xây dựng một hệ thống tìm kiếm lai (Hybrid Search) toàn diện dành cho tài liệu học thuật, sử dụng Elasticsearch làm nền tảng cốt lõi. Bằng cách kết hợp ưu điểm của cả hai phương pháp: tìm kiếm văn bản truyền thống (Keyword Search) để đảm bảo độ chính xác tuyệt đối của các thuật ngữ chuyên ngành, và tìm kiếm vector (Vector Search) để nắm bắt ý nghĩa ngữ cảnh, hệ thống kỳ vọng sẽ khắc phục triệt để các hạn chế của các bộ máy tìm kiếm hiện tại, từ đó mang lại kết quả tra cứu chính xác và tối ưu nhất cho người dùng.

- **Về quản trị và xử lý dữ liệu:** Thiết lập quy trình thu thập, làm sạch và chuẩn hóa khối lượng lớn dữ liệu học thuật thô từ các nguồn mở. Đảm bảo dữ liệu được định dạng tối ưu, sẵn sàng cho việc lập chỉ mục (indexing) và truy xuất ở quy mô hàng triệu bản ghi.
- **Về kiến trúc nền tảng:** Triển khai một hạ tầng tìm kiếm phân tán mạnh mẽ, có khả năng lưu trữ và xử lý đồng thời cả dữ liệu văn bản thuần túy và các biểu diễn không gian vector đa chiều. Hệ thống phải đảm bảo tính sẵn sàng cao và phản hồi truy vấn với độ trễ thấp (low latency).
- **Về thuật toán truy xuất cốt lõi:** Phát triển cơ chế tìm kiếm đa luồng, cho phép thực thi song song các truy vấn dựa trên từ vựng và truy vấn dựa trên ngữ nghĩa. Trọng tâm là xây dựng cấu trúc tự động đánh giá, tính điểm và hợp nhất xếp hạng từ các nguồn kết quả khác nhau để tối ưu hóa độ chính xác.
- **Về tính ứng dụng và đánh giá:** Hoàn thiện giao diện tương tác trực quan, hỗ trợ người dùng thực hiện các thao tác tra cứu và lọc dữ liệu phức tạp. Đồng thời, áp dụng các bộ tiêu chí đánh giá định lượng khoa học để chứng minh hiệu năng của mô hình Tìm kiếm Lai so với các phương pháp truyền thống.

1.3 Phạm vi (Scope)

Nhằm tối ưu hóa hiệu quả thực hiện và bám sát các mục tiêu kỹ thuật cốt lõi, phạm vi nghiên cứu của dự án được giới hạn trong các phân đoạn cụ thể sau:

- **Về nguồn tài nguyên dữ liệu:** Dự án khai thác kho lưu trữ tài liệu học thuật mở arXiv thông qua bộ dữ liệu từ Kaggle. Đây là nguồn tài nguyên tri thức đa ngành quy mô lớn, cung cấp hệ thống thuộc tính phong phú từ nội dung văn bản cho đến các siêu dữ liệu phân loại, tạo cơ sở vững chắc cho việc thực nghiệm các mô hình truy xuất thông tin phức tạp.

- **Về quy mô triển khai giai đoạn giữa kỳ:** Nhóm thực hiện khoanh vùng và xử lý trên một tập dữ liệu con tiêu biểu gồm 100.000 bài báo thuộc lĩnh vực Khoa học máy tính. Việc giới hạn quy mô trong một chuyên ngành cụ thể giúp tối ưu hóa quy trình làm sạch dữ liệu, chuẩn hóa các thuật ngữ chuyên biệt và đảm bảo tính kiểm soát cao trong các kịch bản đánh giá độ chính xác của kết quả tìm kiếm.
- **Về hạ tầng và kỹ thuật thực hiện:** Phạm vi triển khai hạ tầng tập trung vào việc thiết lập hệ thống tìm kiếm phân tán trên hệ sinh thái Elasticsearch phiên bản 8.x. Các cấu hình lõi bao gồm định nghĩa cấu trúc chỉ mục, thiết lập các bộ phân tích từ vựng chuyên sâu và tối ưu hóa hiệu suất nạp dữ liệu quy mô lớn thông qua các phương thức xử lý hàng loạt.
- **Về phương pháp luận và chức năng cốt lõi:** Trọng tâm của giai đoạn hiện tại là hoàn thiện hệ thống tìm kiếm từ vựng dựa trên thuật toán BM25. Hệ thống này được xác lập như một hệ thống cơ sở để làm tiền đề quan trọng giúp nhóm tiến hành đối chứng và đo lường mức độ cải thiện hiệu năng khi tích hợp các công nghệ tìm kiếm nâng cao trong các giai đoạn tiếp theo của đề án.

Chương 2

Dataset (Dữ liệu)

2.1 Nguồn dữ liệu

Dự án sử dụng bộ dữ liệu arXiv từ Kaggle [2] (<https://www.kaggle.com/datasets/Cornell-University/arxiv>), một kho lưu trữ mở chứa các bài báo học thuật do Cornell University duy trì và cập nhật liên tục.

2.2 Quy mô và Đặc điểm

- **Tổng số lượng:** Hơn 2.7 triệu bài báo. Đây là một tập dữ liệu có tính tích lũy cao, được cập nhật liên tục.
- **Dung lượng file gốc:** Hơn 3GB (file JSON dạng line-delimited, mỗi dòng là một bản ghi JSON).
- **Độ phủ:** Hơn 130 lĩnh vực (Computer Science, Physics, Mathematics,...).
- **Tập con sử dụng:** 100,000 bài báo thuộc lĩnh vực Computer Science (`cs.*`).

2.3 Cấu trúc dữ liệu khai thác

Bảng 2.3.1 mô tả cấu trúc các trường dữ liệu được trích xuất và sử dụng trong hệ thống:

Trường	Kiểu gốc	ES Type	Mô tả & Vai trò
id	string	keyword	Mã định danh duy nhất của bài báo trên arXiv
title	string	text	Tiêu đề bài báo – phục vụ tìm kiếm BM25 (boost $\times 2$)
abstract	string	text	Tóm tắt nội dung – phục vụ tìm kiếm BM25
categories	string	keyword	Danh mục phân loại (VD: <code>cs.AI</code> , <code>cs.LG</code>) – phục vụ exact-match filtering
authors	string	text	Danh sách tác giả
year	string	keyword	Năm cập nhật – phục vụ exact-match filtering

Table 2.3.1: Cấu trúc dữ liệu được lập chỉ mục vào *Elasticsearch*

2.4 Quy trình làm sạch dữ liệu (Data Cleaning)

Do file gốc có dung lượng hơn 3GB, việc đọc toàn bộ vào bộ nhớ (RAM) cùng lúc là không khả thi trên hầu hết cấu hình máy tính phổ thông. Nhóm đã áp dụng kỹ thuật **streaming (đọc từng dòng)** để xử lý tuần tự, giúp kiểm soát tốt bộ nhớ ngay cả khi kích thước dữ liệu lên đến hàng GB. Quy trình cụ thể bao gồm:

1. **Đọc tuần tự (Line-by-line streaming):** Mở file gốc và xử lý từng dòng JSON, tránh tải toàn bộ file vào RAM.
2. **Lọc danh mục (Category filtering):** Chỉ giữ lại các bản ghi có trường `categories` chứa tiền tố `cs.` (Computer Science).
3. **Loại bỏ bản ghi lỗi:** Bỏ qua các dòng không parse được (malformed JSON) và các bản ghi thiếu trường `title` hoặc `abstract`.
4. **Chuẩn hóa văn bản:** Loại bỏ ký tự xuống dòng (`\n`) trong `title` và `abstract`, trim khoảng trắng dư thừa.
5. **Trích xuất metadata:** Tách trường `year` từ `update_date` (định dạng YYYY-MM-DD).
6. **Giới hạn mẫu:** Dừng lại sau khi thu thập đủ 100,000 bản ghi hợp lệ.



7. **Xuất file:** Lưu kết quả dưới dạng JSON chuẩn (~350MB) để phục vụ cho bước Bulk Indexing tiếp theo.

Chương 3

Vấn đề và Giải pháp (Problems & Proposed Solutions)

3.1 Problem 1: Sự cứng nhắc của tìm kiếm từ khóa (The Rigidity of Keyword Search)

- **Vấn đề:** Người dùng phải gõ chính xác từ khóa. Các từ đồng nghĩa (synonyms) hoặc cách diễn đạt khác (rephrasing) sẽ trả về 0 kết quả.
- **Giải pháp Elasticsearch:** Cung cấp khả năng Vector Search (kNN) để hiểu "ngữ nghĩa ngữ cảnh" của câu truy vấn thay vì chỉ khớp từng từ.

3.2 Problem 2: AI bỏ sót các thuật ngữ chuyên ngành (AI Misses Specialized Terminology)

- **Vấn đề:** AI hiểu ngữ cảnh tốt nhưng lại kém trong việc tìm kiếm chính xác tên tác giả hoặc các từ viết tắt hiếm gặp.
- **Giải pháp Elasticsearch:** Chạy song song cả hai luồng (Hybrid Search) và tích hợp sẵn thuật toán Reciprocal Rank Fusion (RRF) để tự động tính điểm và gộp kết quả từ AI và Keyword với độ chính xác cao.

3.3 Problem 3: Nhiễu tìm kiếm đa ngành (Search Skewed by Multidisciplinary Noise)

- **Vấn đề:** Một từ khóa có thể có nhiều nghĩa. Việc áp dụng các bộ lọc cứng (năm, danh mục) vào hệ thống AI thường làm phá vỡ cấu trúc tìm kiếm, gây thiếu sót kết quả hoặc độ trễ cao.
- **Giải pháp Elasticsearch:** Tính năng Automatic Optimization filtering giúp tự động chuyển đổi giữa các thuật toán quét (scanning algorithms) dựa trên



độ nghiêm ngặt của bộ lọc, đảm bảo phản hồi nhanh và chính xác.

Chương 4

Cơ sở lý thuyết (Theoretical Background)

4.1 Information Retrieval Pipeline

Với hệ thống truy xuất thông tin (Information Retrieval - IR), thay vì thực hiện thao tác rà soát tuần tự trên toàn bộ cơ sở dữ liệu vốn không khả thi với dữ liệu lớn, hệ thống IR hoạt động dựa trên một quy trình xử lý luồng (pipeline) khép kín. Quy trình này chịu trách nhiệm chuyển đổi khối lượng lớn dữ liệu phi cấu trúc thành các ma trận toán học và cấu trúc lưu trữ tối ưu, bao gồm ba giai đoạn chính:

1. **Ingestion (Thu thập và làm sạch):** Dữ liệu thô từ nhiều nguồn được thu thập, tiến hành chuẩn hóa định dạng và loại bỏ các thành phần nhiễu giúp bóc tách và trích xuất các trường thông tin quan trọng nhằm tạo ra một tập dữ liệu đầu vào sạch, đáp ứng yêu cầu dữ liệu đầu vào cho các bước xử lý tiếp theo.
2. **Indexing (Lập chỉ mục):** Dữ liệu sau khi làm sạch được tổ chức thành các cấu trúc dữ liệu đặc biệt (như Inverted Index đối với văn bản) để tối ưu hóa tốc độ tìm kiếm.
3. **Query Processing (Xử lý truy vấn):** Hệ thống tiếp nhận câu truy vấn từ người dùng, phân tích (tokenize), và đối chiếu với chỉ mục để trả về các kết quả phù hợp nhất dựa trên thuật toán xếp hạng (ranking algorithm).

4.2 Inverted Index và Thuật toán BM25

Trong giai đoạn hiện tại, nhằm đáp ứng yêu cầu tìm kiếm với hiệu năng cao trên các tập dữ liệu lớn, hệ thống sử dụng các công nghệ tìm kiếm văn bản cốt lõi:

- **Inverted Index (Chỉ mục đảo ngược):** Đây là cấu trúc dữ liệu nền tảng của Elasticsearch. Thay vì quét từng tài liệu để tìm từ khóa, Inverted Index lưu trữ một danh sách các từ khóa (terms) và ánh xạ mỗi từ khóa đến danh

sách các tài liệu (documents) chứa nó. Điều này giúp tốc độ tìm kiếm đạt mức gần thời gian thực (near real-time) ngay cả với Big Data.

- **BM25 (Best Matching 25):** Là thuật toán xếp hạng mặc định của Elasticsearch, được phát triển và cải tiến dựa trên nền tảng của mô hình trọng số TF-IDF (Term Frequency - Inverse Document Frequency) [7]. Thuật toán này giúp khai phá văn bản và truy xuất thông tin, đánh giá và xếp hạng độ liên quan của các tài liệu trả về so với câu truy vấn của người dùng dựa trên:
 - **TF (Term Frequency):** Tần suất xuất hiện của từ khóa trong tài liệu. Tần suất càng cao, điểm số càng tăng, nhưng có giới hạn bão hòa để tránh việc lặp từ khóa quá mức.
 - **IDF (Inverse Document Frequency):** Độ hiếm của từ khóa trên toàn bộ tập dữ liệu. Các từ khóa xuất hiện trong ít tài liệu (như thuật ngữ chuyên ngành) sẽ có trọng số cao hơn so với các từ thông dụng (như "the", "a", "is").

4.3 Field Types trong Elasticsearch

Elasticsearch phân biệt rõ ràng hai loại dữ liệu văn bản phục vụ cho các mục đích truy vấn khác nhau [4]. Bảng 4.3.1 tóm tắt sự khác biệt:

Đặc điểm	text	keyword
Phân tích cú pháp	Có (tokenization, lowercasing, stop-word removal)	Không – lưu nguyên bản
Cấu trúc lưu trữ	Inverted Index (ánh xạ term → documents)	Doc Values (ánh xạ document → giá trị gốc)
Loại truy vấn	Full-text search (BM25 scoring)	Exact-match, filtering, aggregation
Ví dụ trường	title, abstract, authors	year, categories, id
Ví dụ truy vấn	“machine learning” → khớp “Machine Learning for...”	“2023” → chỉ khớp chính xác “2023”

Table 4.3.1: So sánh *text* và *keyword* field types trong Elasticsearch

Việc lựa chọn đúng field type cho từng trường dữ liệu là yếu tố then chốt ảnh hưởng trực tiếp đến hiệu năng tìm kiếm và tính chính xác của kết quả trả về.

4.4 Vector Embedding và k-Nearest Neighbors (kNN)

Để giải quyết hạn chế của tìm kiếm từ khóa (không hiểu ngữ nghĩa), hệ thống đề xuất sử dụng các mô hình ngôn ngữ học sâu (như `SentenceTransformers` [6]) để chuyển đổi văn bản thành các vector số thực (dense vectors). Thuật toán k-Nearest Neighbors (kNN) sau đó được áp dụng để tính toán độ tương đồng (ví dụ: Cosine Similarity) giữa vector của câu truy vấn và vector của tài liệu, từ đó tìm ra các kết quả có ngữ nghĩa gần nhất.

4.5 Reciprocal Rank Fusion (RRF)

RRF là một thuật toán xếp hạng (ranking algorithm) kết hợp kết quả từ nhiều phương pháp tìm kiếm khác nhau (ở đây là BM25 và kNN) mà không cần phải chuẩn hóa điểm số (score) của chúng [1]. Công thức RRF tính điểm cho mỗi document d dựa trên vị trí xếp hạng trong từng hệ thống:

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + r(d)}$$

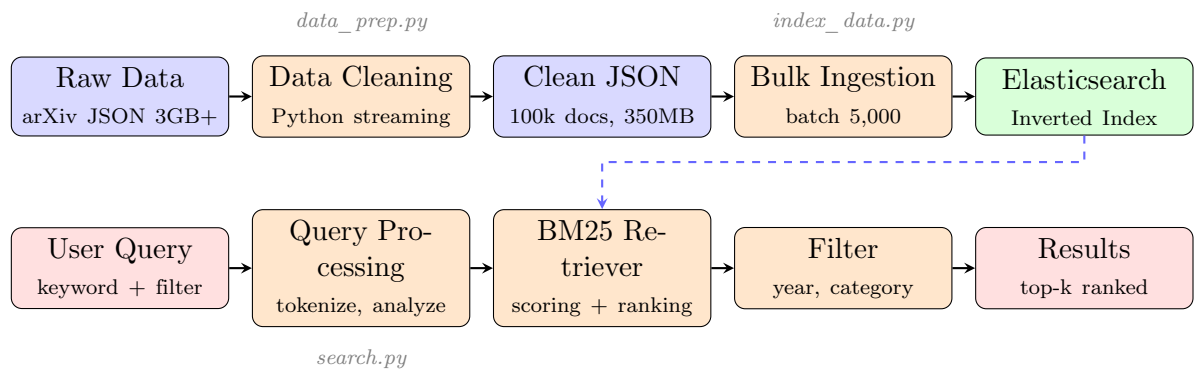
trong đó R là tập các bảng xếp hạng (từ BM25 và kNN), $r(d)$ là vị trí của document d trong bảng xếp hạng r , và k là hằng số điều chỉnh (thường $k = 60$). Công thức này giúp đẩy các tài liệu xuất hiện ở thứ hạng cao trong cả hai phương pháp lên vị trí top đầu, mang lại kết quả tối ưu và chính xác nhất cho hệ thống Hybrid Search.

Chương 5

Kiến trúc hệ thống (System Architecture)

5.1 Sơ đồ luồng dữ liệu (Data Flow)

Quy trình xử lý dữ liệu của hệ thống trong giai đoạn hiện tại được thiết kế theo dạng đường ống (pipeline) tuyến tính, được minh họa trong Hình 5.1.1:



Hình 5.1.1: Sơ đồ luồng dữ liệu (Data Flow Pipeline) của hệ thống

Quy trình bao gồm hai luồng chính:

Luồng nạp dữ liệu (Data Ingestion Pipeline):

1. **Raw Data:** Dữ liệu gốc định dạng JSON từ arXiv (3GB+, line-delimited).
2. **Data Cleaning:** Script Python đọc từng dòng (streaming), lọc bỏ các bản ghi lỗi, và trích xuất tập con 100,000 bài báo thuộc lĩnh vực Computer Science.
3. **Data Ingestion:** Dữ liệu sạch được đóng gói thành các batch 5,000 documents và gửi qua Elasticsearch Bulk API.

4. **Indexing & Storage:** Elasticsearch phân tích văn bản (tokenization) và tạo Inverted Index cho các trường **text**, đồng thời lưu trữ nguyên bản các trường **keyword**.

Luồng truy vấn (Query Pipeline):

1. **User Query:** Người dùng nhập câu truy vấn từ khóa, kèm theo bộ lọc (năm, danh mục) nếu cần.
2. **Query Processing:** Hệ thống phân tích cú pháp (tokenize) câu truy vấn.
3. **BM25 Retrieval:** Đối chiếu với Inverted Index, tính điểm BM25 cho từng document.
4. **Filtering & Ranking:** Áp dụng bộ lọc exact-match và trả về top-k kết quả được xếp hạng theo độ liên quan.

5.2 Kiến trúc hạ tầng (Infrastructure)

5.2.1 Công nghệ sử dụng

- **Python 3.x:** Xử lý dữ liệu (Data Processing) và tương tác với Elasticsearch thông qua thư viện `elasticsearch-py`.
- **Elasticsearch 8.12:** Search Engine cốt lõi, hỗ trợ Inverted Index, BM25 scoring, và filtering [4].
- **Docker & Docker Compose:** Đóng gói và triển khai toàn bộ hạ tầng (Elasticsearch + Kibana) trong các container độc lập [3].
- **Kibana 8.12:** Giao diện quản trị và trực quan hóa dữ liệu, hỗ trợ Dev Tools để thử nghiệm truy vấn.

5.2.2 Mô hình triển khai

Hệ thống được triển khai dưới dạng Single-node cluster trên môi trường Docker cục bộ, sử dụng cấu hình Docker Compose như sau:

```
1 services:
2   elasticsearch:
3     image: elasticsearch:8.12.2
4     environment:
5       - discovery.type=single-node
```

```
6     - xpack.security.enabled=false
7     - "ES_JAVA_OPTS=-Xms2g -Xmx2g"
8     ports:
9     - "9200:9200"
10    volumes:
11    - es_data:/usr/share/elasticsearch/data
12
13    kibana:
14      image: kibana:8.12.2
15      environment:
16      - ELASTICSEARCH_HOSTS=http://elasticsearch:9200
17      ports:
18      - "5601:5601"
```

Program 5.1: *Docker Compose – Elasticsearch & Kibana*

5.2.3 Đặc tính Big Data

Mặc dù chạy trên single-node, kiến trúc bên trong của Elasticsearch vẫn duy trì các khái niệm phân tán như **Index**, **Shard** (phân mảnh dữ liệu) và **Replica** (bản sao). Điều này cho phép hệ thống dễ dàng mở rộng (scale-out) bằng cách thêm các node mới vào cluster khi khối lượng dữ liệu tăng lên mức hàng triệu bài báo mà không cần thay đổi kiến trúc ứng dụng.

5.3 Index Mapping & Bulk Strategy

5.3.1 Mapping Definition

Lược đồ dữ liệu (Mapping) được định nghĩa tường minh để tối ưu hóa hiệu năng cho từng loại truy vấn:

```
1 {
2   "mappings": {
3     "properties": {
4       "id":           { "type": "keyword" },
5       "title":        { "type": "text" },
6       "abstract":     { "type": "text" },
7       "categories":   { "type": "keyword" },
8       "authors":      { "type": "text" },
9       "year":         { "type": "keyword" }
```

```
10     }  
11   },  
12   "settings": {  
13     "number_of_shards": 1,  
14     "number_of_replicas": 0  
15   }  
16 }
```

Program 5.2: *Elasticsearch Index Mapping*

Trong đó:

- Các trường `title`, `abstract`, `authors` được map dưới dạng `text` – Elasticsearch tự động thực hiện phân tích cú pháp (tokenization, lowercasing, stop-word removal) và xây dựng Inverted Index để phục vụ tìm kiếm BM25.
- Các trường `id`, `year`, `categories` được map dưới dạng `keyword` – lưu trữ nguyên bản, không qua phân tích, phục vụ exact-match filtering với tốc độ $O(1)$.

5.3.2 Bulk Indexing Strategy

Thay vì gửi từng tài liệu qua HTTP POST riêng lẻ (rất chậm và tốn tài nguyên mạng), hệ thống sử dụng hàm `helpers.bulk()` của thư viện `elasticsearch-py`. Dữ liệu được gom thành các lô (batch) với kích thước **5,000 documents** mỗi lần gửi, giúp:

- Giảm số lượng HTTP round-trip từ 100,000 xuống còn 20 requests.
- Tối ưu hóa băng thông mạng và giảm tải cho bộ nhớ heap của Elasticsearch.
- Toàn bộ 100,000 documents được index trong vòng vài chục giây.

Chương 6

Hiện trạng triển khai (Current Progress)

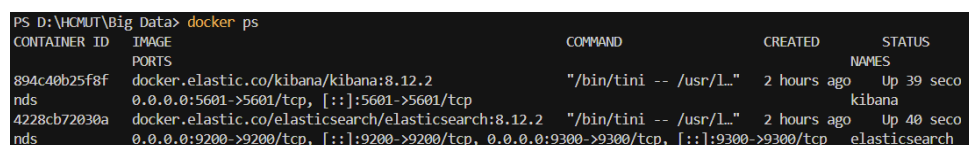
6.1 Các hạng mục đã hoàn thành

Trong giai đoạn đầu của dự án, nhóm đã tập trung xây dựng nền tảng dữ liệu và hệ thống tìm kiếm cơ sở (Baseline Search), cụ thể:

6.1.1 Thiết lập hạ tầng (Infrastructure Setup)

Triển khai thành công cụm Elasticsearch 8.12 và giao diện quản trị Kibana thông qua Docker Compose. Cấu hình bao gồm:

- Elasticsearch single-node cluster với 2GB heap memory.
- Kibana cho phép trực quan hóa dữ liệu và thử nghiệm truy vấn qua Dev Tools.
- Persistent volume đảm bảo dữ liệu không bị mất khi container restart.



CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	NAMES
894c40b25f8f	docker.elastic.co/kibana/kibana:8.12.2	"/bin/tini -- /usr/L..."	2 hours ago	Up 39 seco	kibana
4228cb72030a	docker.elastic.co/elasticsearch/elasticsearch:8.12.2	"/bin/tini -- /usr/L..."	2 hours ago	Up 40 seco	elasticsearch

Hình 6.1.1: Docker containers đang chạy Elasticsearch 8.12 và Kibana

6.1.2 Tiền xử lý dữ liệu (Data Engineering)

Xây dựng script Python (data_prep.py) để xử lý file dữ liệu gốc hơn 3GB bằng kỹ thuật streaming (đọc từng dòng). Kết quả:

- Trích xuất thành công **100,000 bài báo** thuộc lĩnh vực Computer Science.
- File output (arxiv_cs_100k_clean.json): ~350MB, định dạng JSON chuẩn.
- Thời gian xử lý: vài phút trên máy cá nhân.

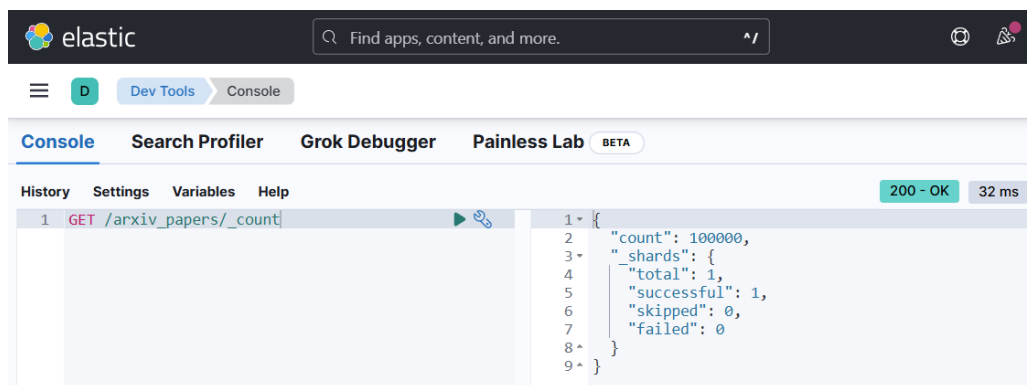
6.1.3 Lập chỉ mục (Bulk Indexing)

Thiết kế Index Mapping tường minh và sử dụng Bulk API để đẩy toàn bộ dữ liệu vào Elasticsearch (`index_data.py`):

- 100,000 documents được index thành công với batch size 5,000.
- Kiểm chứng qua Kibana Dev Tools (Hình 6.1.2):

```
1 GET /arxiv_papers/_count
2 // Response:
3 {
4   "count": 100000,
5   "_shards": {
6     "total": 1,
7     "successful": 1,
8     "skipped": 0,
9     "failed": 0
10  }
11 }
```

Program 6.1: Kết quả kiểm tra số lượng documents đã index



Hình 6.1.2: Kibana Dev Tools – Kết quả xác nhận 100,000 documents đã được index

6.1.4 Truy vấn cơ sở (Baseline BM25 Search)

Hiện thực module tìm kiếm từ khóa (`search.py`) sử dụng thuật toán BM25 với các tính năng:

- Tìm kiếm đồng thời trên `title` (boost $\times 2$) và `abstract` sử dụng `multi_match` query.
- Hỗ trợ lọc theo `year` thông qua `bool` filter (exact-match).
- Kết quả được xếp hạng theo BM25 score, trả về top-k documents.

Ví dụ truy vấn BM25 kết hợp filter theo năm và kết quả thực tế từ hệ thống:

```
1 GET /arxiv_papers/_search
2 {
3   "query": {
4     "bool": {
5       "must": [{
6         "multi_match": {
7           "query": "deep learning transformer",
8           "fields": ["title^2", "abstract"],
9           "type": "best_fields"
10        }
11      }],
12      "filter": [{ "term": { "year": "2023" } }]
13    }
14  },
15  "size": 3,
16  "_source": ["title", "year", "categories"]
17 }
```

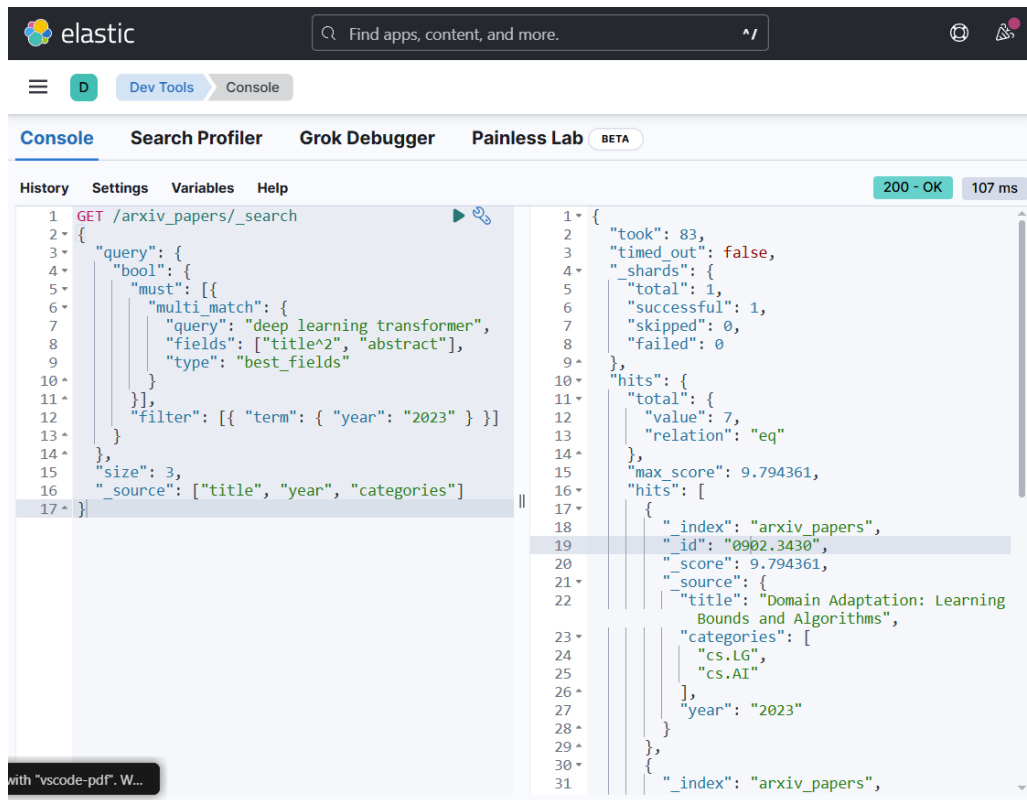
Program 6.2: Truy vấn BM25 “deep learning transformer” + filter year=2023

```
1 {
2   "took": 73,
3   "hits": {
4     "total": { "value": 7 },
5     "max_score": 9.794,
6     "hits": [
7       {
8         "_score": 9.794,
9         "_source": {
10           "title": "Domain Adaptation: Learning Bounds
11                  and Algorithms",
12           "categories": ["cs.LG", "cs.AI"],
13           "year": "2023"
14         }
15       }
16     ]
17   }
18 }
```

```
15     },
16     {
17         "_score": 7.294,
18         "_source": {
19             "title": "rFerns: An Implementation of the
20                     Random Ferns Method for
21                     General-Purpose Machine Learning",
22             "categories": ["cs.LG", "stat.ML"],
23             "year": "2023"
24         }
25     }
26 ]
27 }
28 }
```

Program 6.3: *Kết quả trả về từ Elasticsearch (trích lược)*

Kết quả cho thấy hệ thống BM25 đã hoạt động chính xác: trả về 7 kết quả phù hợp với thời gian phản hồi **73ms**, các tài liệu được xếp hạng theo BM25 score giảm dần, và bộ lọc year: 2023 được áp dụng đúng.



Hình 6.1.3: Kibana Dev Tools – Kết quả tìm kiếm BM25 với filter

6.2 Các hạng mục tiếp theo (Next Steps)

Dựa trên nền tảng đã xây dựng, nhóm sẽ tiếp tục triển khai các tính năng nâng cao để hoàn thiện hệ thống Hybrid Search:

- **Vector Embedding:** Sử dụng mô hình AI (pretrained models từ HuggingFace [6]) để chuyển đổi nội dung Title và Abstract thành dense vectors, bổ sung trường dense_vector vào Index Mapping.
- **kNN & Hybrid Search:** Cấu hình tìm kiếm vector (kNN) trên Elasticsearch và tích hợp thuật toán RRF [1] để chạy song song cùng BM25, hợp nhất kết quả từ hai luồng tìm kiếm.
- **Giao diện người dùng (UI):** Xây dựng ứng dụng web bằng Streamlit để người dùng có thể tương tác trực quan với hệ thống (nhập query, chọn chế độ



tìm kiếm, xem kết quả).

- **Đánh giá mô hình (Evaluation):** Xây dựng bộ truy vấn thử nghiệm và áp dụng các thang đo chuẩn trong Information Retrieval (như NDCG, MRR) để chứng minh tính hiệu quả của Hybrid Search so với Keyword Search thuần túy.

Chương 7

Kết luận tạm thời

7.1 Tóm tắt kết quả đạt được

Tính đến thời điểm báo cáo giữa kỳ, nhóm đã hoàn thành toàn bộ Phase 1 của dự án, bao gồm:

- Triển khai thành công hạ tầng Elasticsearch 8.12 + Kibana trên Docker.
- Xử lý và làm sạch tập dữ liệu 3GB+ thành subset 100,000 bài báo CS bằng kỹ thuật streaming.
- Lập chỉ mục toàn bộ 100,000 documents vào Elasticsearch qua Bulk API.
- Xây dựng và kiểm thử thành công hệ thống tìm kiếm BM25 baseline với khả năng kết hợp filter.

7.2 Đánh giá tính khả thi

Dự án hiện tại đang bám sát tiến độ mà nhóm đề ra. Việc thu hẹp phạm vi xuống 100,000 bài báo cho giai đoạn Midterm giúp nhóm kiểm soát tốt tài nguyên phần cứng (RAM/CPU) khi chạy Elasticsearch trên Docker cục bộ, đồng thời vẫn thể hiện được tư duy xử lý Big Data thông qua Bulk API và kiến trúc phân tán (Sharding).

Hệ thống BM25 baseline đã chạy ổn định và cho kết quả trả về hợp lý trên nhiều loại truy vấn khác nhau (general search, filtered search, specialized terminology). Đây là nền tảng vững chắc để tiến hành đối chứng (benchmarking) khi tích hợp thêm Vector Search ở giai đoạn tiếp theo.

7.3 Bước tiếp theo

Ngay sau khi hoàn thành báo cáo giữa kỳ, nhóm sẽ tiến hành:

1. Nghiên cứu thư viện **SentenceTransformers** [6] để tạo Vector Embeddings cho 100,000 bài báo.



2. Mở rộng Index Mapping để hỗ trợ trường `dense_vector` cho kNN search.
3. Tích hợp thuật toán Reciprocal Rank Fusion (RRF) [1] để kết hợp kết quả BM25 và kNN.
4. Xây dựng giao diện Streamlit và bộ đánh giá NDCG/MRR.

Tài liệu tham khảo

- [1] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759. ACM, 2009.
- [2] Cornell University. arxiv dataset, 2024. Accessed: 2026-03-15.
- [3] Docker Inc. Docker compose overview, 2024. Accessed: 2026-03-15.
- [4] Elastic N.V. Elasticsearch reference [8.12], 2024. Accessed: 2026-03-15.
- [5] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [6] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3982–3992. Association for Computational Linguistics, 2019.
- [7] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.